

演算法報告

楊文德

1. 程式介面與介紹(終端)

```
-----  
Please enter the size of an array : 10000  
Merge Sort Time : 0.009 seconds  
Insertion Sort Time : 0.106 seconds  
-----
```

程式需要手動操作的部分只有一開始需輸入陣列大小，之後等待一段時間就可以知道 merge sort 與 insertion sort 分別要跑多久時間。

2. 不同陣列大小下的表現

Please enter the size of an array : 10 Merge Sort Time : 0.000 seconds Insertion Sort Time : 0.000 seconds	Please enter the size of an array : 100 Merge Sort Time : 0.000 seconds Insertion Sort Time : 0.000 seconds
Please enter the size of an array : 1000 Merge Sort Time : 0.002 seconds Insertion Sort Time : 0.001 seconds	Please enter the size of an array : 5000 Merge Sort Time : 0.010 seconds Insertion Sort Time : 0.027 seconds
Please enter the size of an array : 10000 Merge Sort Time : 0.008 seconds Insertion Sort Time : 0.111 seconds	Please enter the size of an array : 50000 Merge Sort Time : 0.073 seconds Insertion Sort Time : 2.845 seconds
Please enter the size of an array : 100000 Merge Sort Time : 0.111 seconds Insertion Sort Time : 11.218 seconds	Please enter the size of an array : 300000 Merge Sort Time : 0.318 seconds Insertion Sort Time : 99.687 seconds
Please enter the size of an array : 500000 Merge Sort Time : 0.490 seconds Insertion Sort Time : 286.260 seconds	Please enter the size of an array : 1000000 Merge Sort Time : 0.962 seconds Insertion Sort Time : 1131.388 seconds

3. 時間複雜度分析

一、Insertion sort

```
void insertion(vector<int> arr, int size)
{
    double end_time, start_time = clock();
    for(int i=1;i<size;i++)
    {
        int key = arr[i];
        int j = i-1;
        while(j >= 0 && arr[j] > key)
        {
            arr[j+1] = arr[j];
            j--;
        }
        arr[j+1] = key;
    }
    end_time = clock();
    cout << fixed << setprecision(3) << "Insertion Sort Time : " << (end_time - start_time) / (double)1000 << " seconds" << endl;
}
```

撇除掉 clock 函數，只討論演算法本身，演算法的 while 迴圈會跑 $n-1$ 次，每一次都會往前搜尋數字，遇到比當前數字大的就交換，遇到小的就停住，所以按照情況我們可以分成兩個條件討論：

- **Best : $O(n)$** $\Rightarrow$ 當資料為 $1, 2, 3, \dots, n$ 時，僅需比較 n 次就可完成排序。另一方面，當陣列越接近排序過後的樣子，這時的 **insertion sort** 將會很高效。
- **Worest : $O(n^2)$** $\Rightarrow$ 當資料排序為由大到小時，那第 i 位置的元素將會與前面 $i-1$ 個元素依序比較，因此總步驟數為

$$0 + 1 + 2 + \dots + (n-1) = \frac{n(n-1)}{2} = \frac{n^2-n}{2} = O(n^2)$$

從數據上我們可以從差距得出：

當我們陣列大小分別是 10000 與 100000 時，我們分別跑的時間為

0.111 與 11.218 秒，我們可以發現兩者差了快 100 倍，

陣列大小增加 10 倍，但時間增長 100 倍，由此可呼應時間複雜度為 $O(n^2)$ 的關點。

二、Merge sort

```
void merge_divide(vector<int> &arr, int left, int right)
{
    int mid = left + (right - left) / 2;
    if (left < right)
    {
        merge_divide(arr, left, mid);
        merge_divide(arr, mid + 1, right);
        merge_combine(arr, left, mid, right);
    }
}

void merge_combine(vector<int> &arr, int left, int mid, int right)
{
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int *L = new int[n1 + 1];
    int *R = new int[n2 + 1];

    L[n1] = INT_MAX;
    R[n2] = INT_MAX;

    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;

    while (i < n1 || j < n2)
    {
        if (L[i] <= R[j])
        {
            arr[k] = L[i];
            i++;
        }
        else
        {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    delete[] L;
    delete[] R;
}
```

接下來換討論 merge sort 的時間複雜度，由最上面的分化中我們可以看到我是用中間值為界分成兩部分，基本上這步驟會進行 $\log_2 n$ 次，然後之後合併又會進行 $\log_2 n$ 次(倆倆合併)，但是每一次合併又會跑三個迴圈，總計一次合併共要跑 $2n$ 個元素。

將左右陣列的值複製進動態陣列(n)

比較左右動態陣列的值，將最小的值放入原本矩陣達到排序的目的(n)

因此我們可以得知 merge sort 大致上需要跑 $\log_2 n + 2n \log_2 n = O(n \log_2 n)$ 次。

從數據上我們可以從差距得出：

```
Please enter the size of an array : 100000000
Merge Sort Time : 236.010 seconds
```

1M 筆資料需要 0.962 秒

100M 筆資料需要 236.01 秒

按照 $O(n \log_2 n)$ 的理論可以得知兩者理應差 664 倍的時間，但時間只差 245 倍而已，這到底是為什麼呢？

推測:

我大膽地猜測可能的原因如下:

1. 電腦有設置一個臨界值，當數據處理超過某一個數量時會轉換到其他的運算模組，這樣就不只受到演算法的影響。
2. 我是從演算法的時間複雜度直接下手，但是忽略的其中許多步驟(係數等等的)，如果是小數量的可能看不太出來，一旦數量像滾雪球那般越來越大時可能會造成難以估量的後果。
3. 我寫錯了…